

Instituto Tecnológico y de Estudios Superiores de Occidente

Non-Relational Database



Social Media Analytics Tool

Presenting

Stephanie Borrego Arroyo
Hannah Chenoa Puente Rosales
Luis Fernando Del Real Vázquez

Teacher: Leobardo Ruiz Rountree

Date: 04/12/2025

Content

Social Media Analytics tool	1
Project Name	3
Introduction	3
Project Scope	3
Functional Requirements	3
Cassandra	5
Data Models	6
- Interactions Table: Stores all user engagement events as time-series records optimized for high-throughput inserts and chronological queries.	6
DGraph Data Model Design	7
1. Graph Description: Nodes and Edges (Schema)	7
Node Types	7
Edge Predicates & Facets	8
ChromaDB Data Model Design	11
Table Structure	13
Primary Table: interaction_logs	13
Queries	15
MongoDB:	15
Dgraph:	15
Cassandra:	17
ChromaDB:	18
Indexes and Optimizations	19
MongoDB:	19
Performance Justification	20
Cassandra:	20
Dgraph:	20
Performance Justification	21
Chromadb:	21
Conclusion	22
Sources of Information	22

Project Name

Social Media Analytics Tool

Introduction

This project details the design and implementation of a Social Media Analytics Tool built to monitor user activity, analyze social networks, and track engagement trends across multiple platforms. This system leverages the strengths of three distinct NoSQL databases: MongoDB, Cassandra, and Dgraph.

The primary goal is to demonstrate a robust, scalable, and secure data architecture that can handle diverse data types, including user profiles, time-series events, and complex social relationships, while meeting all defined functional requirements.

Project Scope

The objective of this project is to design and implement a robust, scalable, and secure data architecture for a Social Media Analytics Tool. This system is built to monitor user activity, analyze complex social networks, and track engagement trends across multiple platforms.

Core Functionality

User Management (MongoDB)

- User profile creation, update, and deletion
- Authentication and account management
- Public profile viewing
- Social media account linking (Instagram, LinkedIn, Twitter/X)
- Account status management (active, suspended, deleted)

Activity Logging (Cassandra)

- Real-time event logging for all user interactions

- Time-series data storage
- Activity aggregation by time windows
- Recent activity feed queries

Relationship Tracking (Dgraph)

- Follow/follower relationship mapping
- Social interaction tracking (likes, comments, shares)
- Mutual connection identification
- Influencer detection and ranking
- Community cluster analysis
- Relationship strength scoring

Semantic Search & AI Analytics (ChromaDB)

- Vector-based semantic search
- Sentiment analysis integration
- Topic extraction and keyword analysis

Technical Implementation

- Database schema design and optimization
- Indexing strategies for performance
- Cross-database data integration via shared IDs (user_id, post_id)
- Query optimization for each database type
- Security implementation (password hashing, data isolation)
- Analytics dashboard with unified data visualization

Functional Requirements

MongoDB:

1. The system shall allow users to create new profiles with all required information fields: full name, username, email, password, age, gender.
2. The system shall allow users to update profile information and linked social accounts.
3. The system shall validate that both the username and email are unique before creating a profile.
4. The system shall allow saving different profile information for each social media that they want to analyze.
5. The system shall store a timestamp for account creation and last modification.

6. The system shall enable users to view the public profile information of other users.
7. The system shall allow users to delete their user accounts permanently.
8. The system shall store the account status to handle deactivated or deleted accounts.
9. The system shall ensure that sensitive user data (passwords) is encrypted in storage.

Dgraph:

1. The system shall track user relationships, such as followers and following connections, as directed edges between user nodes.
2. The system shall represent interactions (likes, comments, shares) as graph edges connecting users and content nodes.
3. The system shall allow visualization of the relationship graph, including user nodes and their interaction edges.
4. The system shall update graph edges in real time when users follow, like, comment, or share.
5. The system shall allow users to retrieve their followers and following lists through graph queries.
6. The system shall retrieve mutual connections between two users by computing intersecting follow edges.
7. The system shall compute and store a relationship strength score between two users based on interaction frequency.
8. The system shall analyze information propagation paths for specific posts or topics through the network graph.

Cassandra:

1. The system shall log all user activities, including posts, comments, likes, follows, shares, and their timestamps.
2. The system shall record the device type and session ID associated with each user activity.
3. The system shall support querying the activity time range or activity type.
4. The system shall aggregate the total number of activities per hour, day, and month to support time-series analytics.
5. The system shall identify and flag abnormal activity patterns, such as sudden spikes or drops in engagement.

6. The system shall compute inactivity periods for each user by detecting time gaps between activity timestamps.
7. The system shall support queries for activity reports grouped by time windows (daily, weekly, monthly).
8. The system shall store time-based indexes to optimize read performance for chronological queries.

Chromadb:

1. The system shall store social media post content as high-dimensional vector embeddings.
2. The system shall support semantic similarity search using natural language queries independent of keyword matching.
3. The system shall return the top N most semantically relevant posts for a given query based on cosine similarity scores.
4. The system shall automatically generate embeddings using the 'all-MiniLM-L6-v2' sentence transformer model without external API dependencies.
5. The system shall link vector embeddings to structured data in MongoDB and Dgraph via 'post_id' metadata.
6. The system shall enable sentiment analysis by retrieving contextually relevant posts for brand monitoring queries.
7. The system shall support keyword extraction by providing a semantically coherent corpus of topic-related documents for NLP processing.

Analytics and Reporting

1. The system shall provide engagement metrics per user and per post (likes, shares, comments, and total interactions).
2. To ensure consistent functionality across all integrated platforms, the system's analysis must be limited to posts.
3. The system shall allow filtering analytics by date range, user, or post category.
4. The system shall identify top-performing users and posts based on engagement scores.
5. The system shall calculate and visualize follower growth and decline over time.
6. The system shall display dashboards with graphs showing engagement trends and user activity levels.

7. The system shall generate automated reports summarizing key metrics (engagement rate).
8. The system shall provide metrics for engagement drop-offs.
9. The system shall integrate data from MongoDB, Dgraph, and Cassandra for unified analytics visualization.
10. The system shall update dashboards in near real-time when new activity or relationship data is logged.

Functional Requirements Modifications

Removed Requirements:

From Cassandra Module:

- **Original FR 22 (now removed):** “The system shall identify and flag abnormal activity patterns, such as sudden spikes or drops in engagement.”
 - **Reason for removal:** Implementing robust anomaly detection algorithms (statistical analysis, threshold definitions, machine learning models) was beyond the scope of a database design project. This feature requires complex pattern recognition that belongs to a separate analytics layer rather than the database itself.
- **Original FR 23 (now removed):** “The system shall compute inactivity periods for each user by detecting time gaps between activity timestamps.”
 - **Reason for removal:** While theoretically possible, computing inactivity periods requires complex temporal gap analysis across large time-series datasets. Given Cassandra's lack of server-side aggregation functions, this would require significant client-side processing logic that diverted focus from core database functionality.

Test Data Specifications

The system was tested with the following demonstration-scale data:

- **MongoDB:** 22 users, 22 profiles
- **Dgraph:** 22 user nodes, 48 post nodes, 116 total edges (38 follows, 42 likes, 18 comments, 18 shares)
- **Cassandra:** 78 activity logs spanning 1.5 months of simulated activity

- **ChromaDB:** 48 vectorized posts with 384-dimensional embeddings

Design

MongoDB:

MongoDB, a NoSQL document database, was chosen for the User Management module due to its flexible schema and efficient handling of profile-related data. The design is split into two collections to enhance security and read performance.

Dgraph:

Dgraph was decided to model the complex, many-to-many relationships between users, which is its primary strength. This is ideal for fulfilling the “Relationship Tracking” functional requirements (FR 10-19), such as finding mutual connections, influencers, and community clusters.

ChromaDB:

ChromaDB was selected as the **Vector Database** to power the “Innovation” modules of the Social Media Analytics Tool: Semantic Search, Sentiment Analytics, and Topic Extraction. Unlike traditional keyword-based search (which fails on synonyms or context), ChromaDB stores data as **high-dimensional vectors (embeddings)**. This allows the system to find posts that are *conceptually* similar to a user’s query, even if they don’t share the same words.

Cassandra

Cassandra was chosen for the **Activity Logging and Interaction Tracking** module due to its high write throughput, horizontal scalability, and ability to handle time-series data efficiently.

Social platforms generate continuous streams of interactions, likes, comments, shares, and other engagement events. Cassandra’s architecture is optimized for exactly this type of workload.

Data Models

MongoDB Data Model Design

The MongoDB data model is designed using a two-collection approach (users and profiles) to maximize security and performance.

- **Security:** The users collection stores only private, sensitive data required for authentication and account management (email, password_hash). This directly supports FR 9 by ensuring sensitive data is isolated and not exposed during public profile lookups.
- **Performance:** The profiles' collection stores all public-facing information (full_name, bio, linked_social_accounts). This allows the application to satisfy FR 6 (view public profile) by reading a single document. The _id of a document in profiles is identical to the _id of its corresponding document in users, creating a fast 1:1 relationship.

Users Collection:

- This collection is designed for **security isolation** and **authentication speed**. It stores the minimum private data necessary for account management.

```
{
  "_id": ObjectId("675a8a5b..."), // Unique ID, shared 1:1 with 'profiles' collection
  "username": "hannah1",         // Used for login and primary lookup
  "email": "hannah@example.com", // Used for login and unique identification
  "password_hash": "$2b$12$...",  // Secure hash of the user's password
  "account_status": "active",     // Status (e.g., active, suspended)
  "timestamps": {
    "created_at": ISODate("..."), // Account creation date
    "modified_at": ISODate("...") // Last modification date
  }
}
```

- **Index Command:** db.users.createIndex({ "email": 1 }, { unique: true })
- **Purpose:** Enforces data integrity and makes authentication queries fast.

- **Index Command:** `db.users.createIndex({ "username": 1 }, { unique: true })`
- **Purpose:** Enforces uniqueness and accelerates login lookups.
- **Index Command:** `db.users.createIndex({ "account_status": 1 })`
- **Purpose:** Supports efficient administrative queries, like finding all suspended users.

Profile Collection:

- This collection stores all **public-facing information** for a user. It is optimized for display, comparison, and analysis. It utilizes the **Document Splitting pattern** for performance.

```
{
  "_id": ObjectId("675a8a5b..."), // ID identical to 'users' for fast 1:1 lookup
  "username": "hannah1",         // Primary field for public profile search
  "full_name": "Hannah Chenoa",
  "age": 20,
  "gender": "female",
  "bio": "...",
  "profile_picture_url": "...",
  "linked_social_accounts": [     // Embedded array for holding social platform data
    {
      "platform_name": "Instagram",
      "username": "hannah7chenoa",
      "url": "https://www.instagram.com/hannah7chenoa/"
    },
    {
      "platform_name": "LinkedIn",
      "username": "Hannah Chenoa Puente Rosales",
      "url": "https://www.linkedin.com/in/hannah-chenoa-puente-rosales/"
    }
  ]
}
```

- **Index Command:** `db.profiles.createIndex({ "username": 1 }, { unique: true })`
- **Purpose:** Essential for public profile performance, allowing instantaneous lookup using the username.
- **Index Command:** `db.profiles.createIndex({ "linked_social_accounts.platform_name": 1 })`
- **Purpose:** This multikey index is specifically designed to optimize the aggregation pipeline below. It enables the database to efficiently scan and group documents based on the embedded platform names.

Aggregation Pipeline:

- Its purpose is to understand the usage of different social platforms by showing which is most linked by users in the app, directly supporting analytics and reporting requirements

```
db.profiles.aggregate([
  { $unwind: "$linked_social_accounts" },
  {
    $group: {
      _id: "$linked_social_accounts.platform_name",
      total_users: { $sum: 1 }
    }
  },
  { $sort: { total_users: -1 } }
])
```

Example output:

```
[
  { "_id": "Instagram", "total_users": 5 },
  { "_id": "LinkedIn", "total_users": 2 },
  { "_id": "X", "total_users": 1 }
]
```

Dashboard: Luis Fernando

User Profile (Mongo) | Activity Logs (Cassandra) | Network & AI (Dgraph/Chroma)

User Profile (MongoDB)

Basic Information

Username: luis_fernando

Gender: male

Full Name: Luis Fernando

Bio: fernandooooo

Age: 25

Edit Profile

Bio

fernandooooo

Gender

male

Save Changes

Linked Social Accounts

	platform_name	handle	followers	posts	profile_url	last_updated
0	Instagram	@fer	20	4		2025-11-28 05:27:33

+ Add New Social Account

Platform

Instagram

Handle

@username

Followers

0

-

+

Posts

0

-

+

Profile URL (optional)

https://...

Add Account

Remove Linked Account

Select account to remove

Instagram — @fer

Delete Selected Account

Platform Popularity (MongoDB)



MongoDB analytics and profile management.

Delete Account

⚠ This action is permanent. Your account and all related data will be deleted.

☐ I understand and want to delete my account permanently.



1. Graph Description: Nodes and Edges (Schema)

The graph schema is designed to model all relationships and interactions using a **Directed Labeled Graph** approach. Unlike relational models that require intermediate join tables, interactions are stored directly as edges between nodes, with metadata (timestamps, scores) stored as **Facets** on those edges.

Node Types

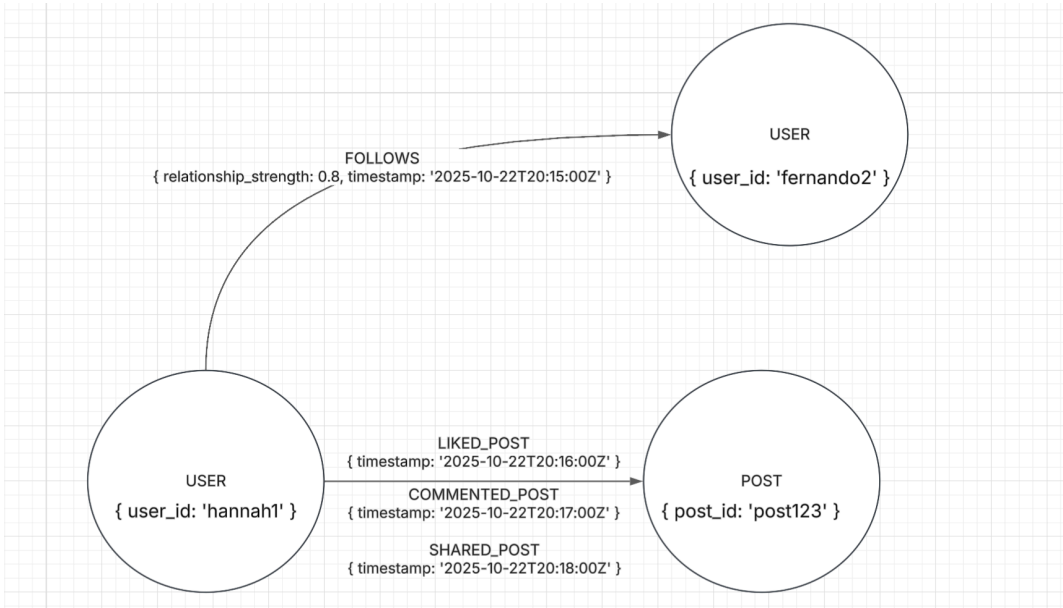
- **Type User:** The central entity of the social graph.
 - user_id (string): Unique identifier linking this node to the MongoDB users collection.
 - FOLLOWS ([uid]): List of edges pointing to other users.
 - LIKED_POST ([uid]): List of edges pointing to posts.
 - COMMENTED_POST ([uid]): List of edges pointing to posts.
- **Type Post:** Represents content entities.
 - post_id (string): Unique identifier for the content.
 - POSTED_BY ([uid]): Reverse edge linking back to the author.

Edge Predicates & Facets

These predicates define the directed relationships. We utilize Dgraph **Facets** to store interaction metadata directly on the edge, optimizing traversal performance.

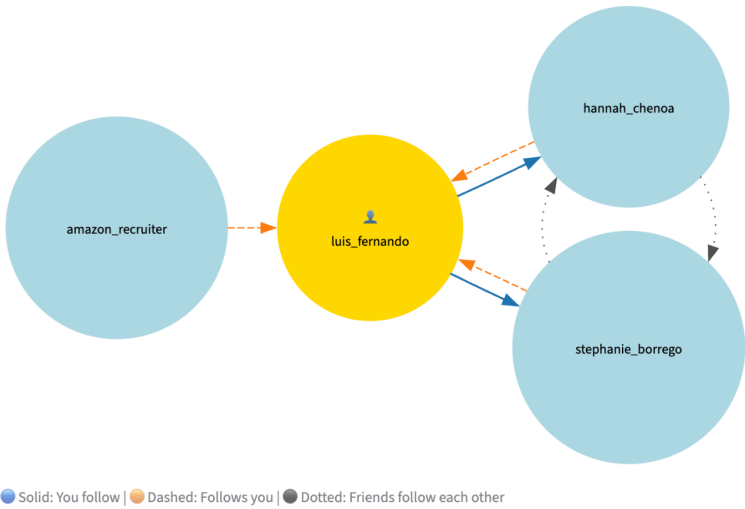
Predicate	Direction	Description	Facets (Edge Attributes)
FOLLOWS	User ->User	Defines a subscription relationship (FR 10).	timestamp: (datetime) When the follow occurred. relationship_strength: (float) A computed score (0.0-5.0) based on

			interaction frequency (FR 16).
LIKED_POST	User ->Post	Represents a "Like" interaction (FR 11).	timestamp: (datetime) Essential for real-time feeds and analytics.
COMMENTED_POST	User ->Post	Represents a "Comment" interaction.	timestamp: (datetime)
SHARED_POST	User ->Post	Represents a "Share" interaction.	timestamp: (datetime)



Graphic Representation (DGraph):

Relationship Map: luis_fernando



Network Insights

Mutual Friends

2

View Mutals

- hannah_chenao
- stephanie_borrego

Communities

Triangles (You ↔ A ↔ B)

Cluster 1: hannah_chenao, luis_fernando, stephanie_borrego

Cluster 2: amazon_recruiter, luis_fernando, stephanie_borrego

User Engagement Metrics

Start Date

2025/10/31

End Date

2025/11/30

Engagement Analysis (30 days)

Total Posts

5

Total Likes

4

Total Comments

2

Overall Engagement

8

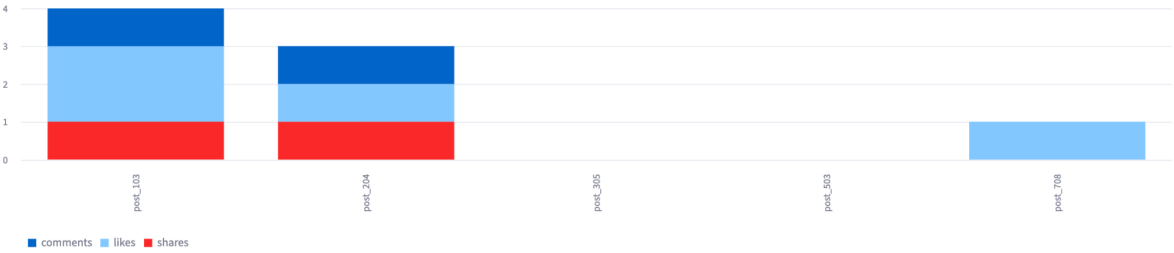
Export Report

Download Summary Report (CSV)

Engagement per Post

	post_id	likes	comments	shares	score
3	post_103	2	1	1	7
4	post_204	1	1	1	6
0	post_708	1	0	0	1
1	post_305	0	0	0	0
2	post_503	0	0	0	0

Engagement Distribution



Engagement Drop-off Analysis (30-day vs. Prior 30-day)

Overall Engagement Change

Current Score: 14

New Activity

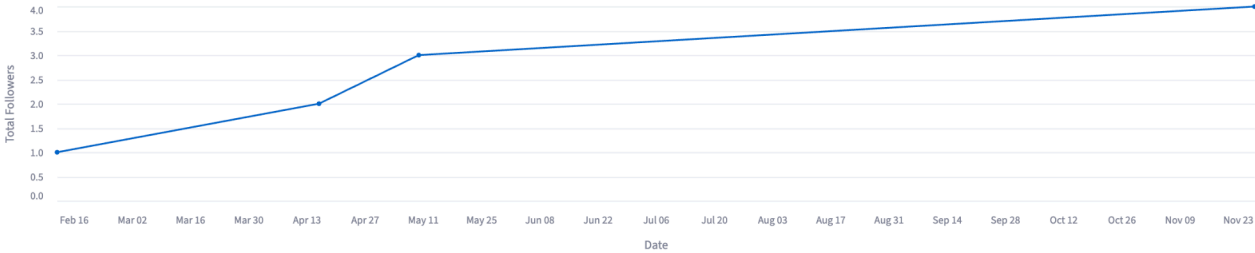
Detailed Comparison

Current Period (30 days): 14 (from 2025-10-31 to 2025-11-30)

Previous Period (30 days): 0 (from 2025-09-30 to 2025-10-30)

Engagement level stable.

Follower Growth Over Time



Information Propagation Path

Source User

luis_fernando

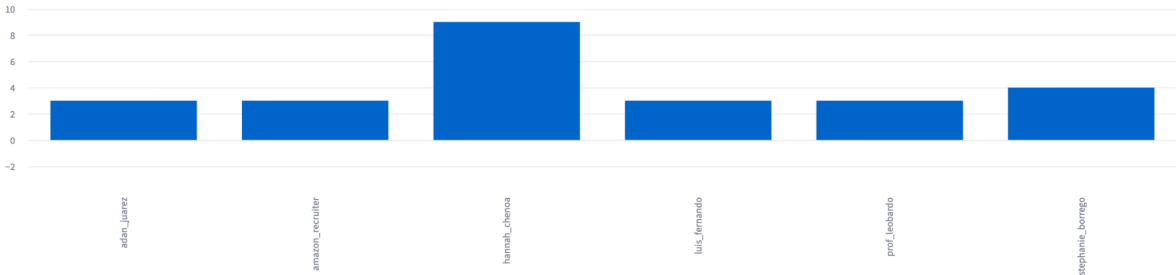
Target User

amazon_recruiter

Trace Path

Path Found: luis_fernando -> stephanie_borrego -> amazon_recruiter

Top Influencers

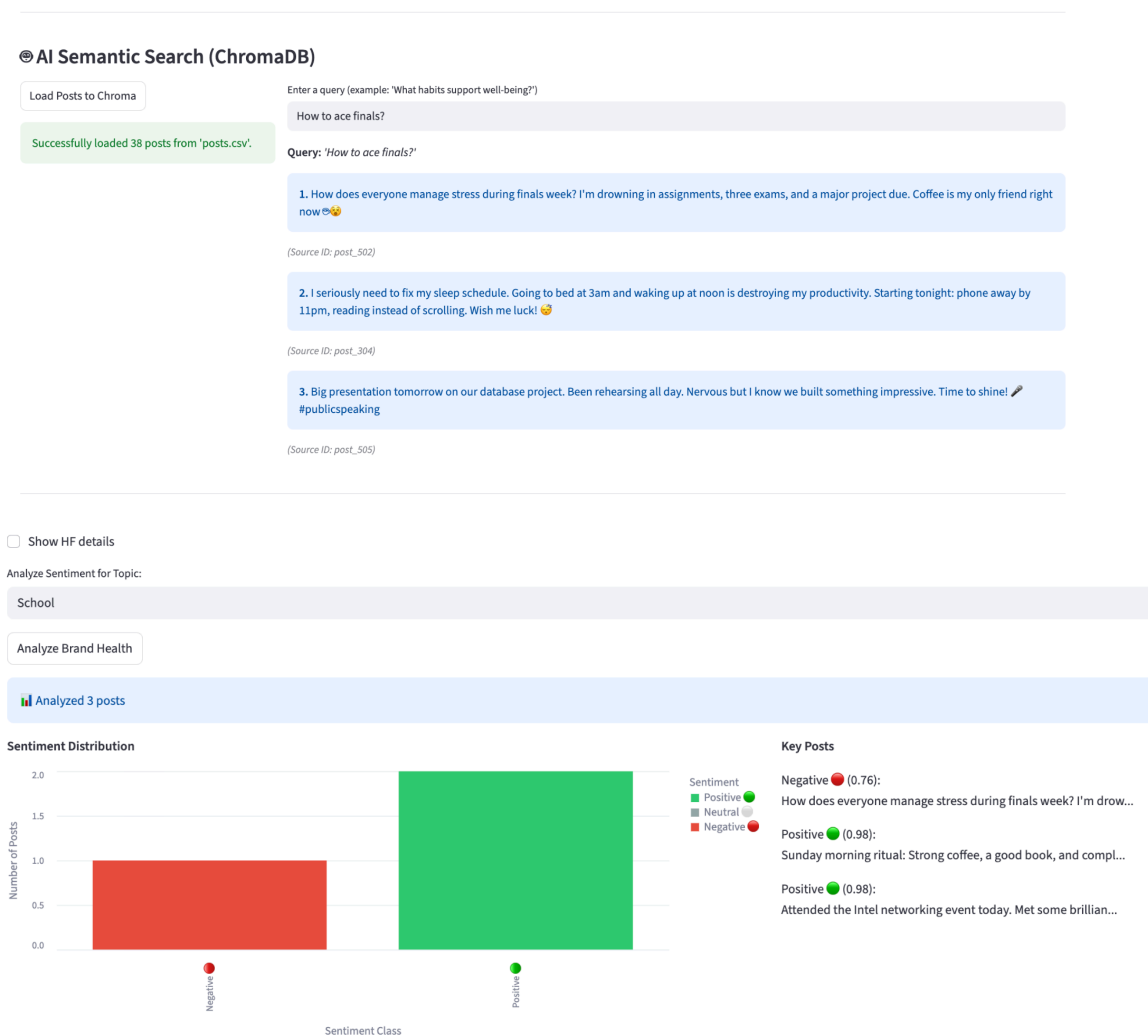


ChromaDB Data Model Design

Textual Representation (Collection Schema): The data model in ChromaDB is flat and collection-based. We define a single collection named `social_posts_project`. Each item in this collection consists of three parallel arrays:

- Documents (Content): The raw text content of the social media post (“I love hiking in the mountains”).
- IDs (Metadata): The unique `post_id` (“post_102”) that links this vector back to the structured data in MongoDB and Dgraph.
- Embeddings (Vector): A high-dimensional float array automatically generated by Chroma's default embedding model upon insertion.

Graphic Representation (DGraph):

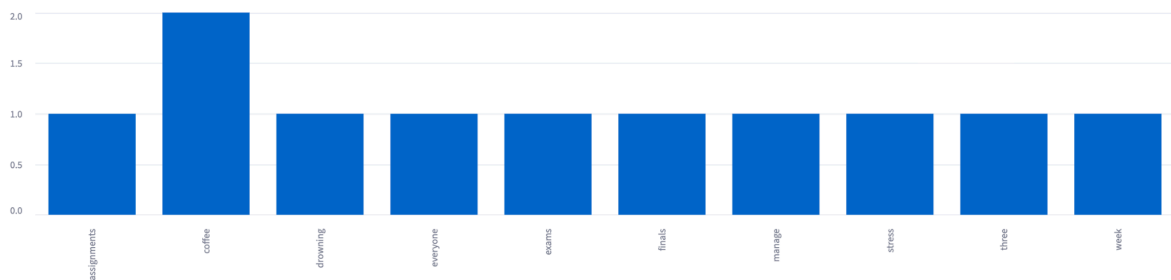


💡 Innovation: Topic Keyword Extractor

Find Key Topics in Posts about:

School

Extract Keywords



Cassandra Data Model Design

Table Description: Wide-Column Time-Series Schema

Cassandra is used to store high-volume interaction events (likes, comments, shares, etc.) using a Wide-Column Time-Series Model.

Instead of graph edges or relational joins, Cassandra organizes data by partition keys and cluster-ordered timestamps, enabling extremely fast writes and chronological reads.

All interaction events are stored in a single denormalized table (interaction_logs) optimized for analytics dashboards and recent-activity queries.

Table Structure

Primary Table: interaction_logs

Field	Type	Description
user_id	TEXT	Partition key; groups all events by user
timestamp	TIMESTAMP	Clustering key; sorted DESC for fast “most

		recent activity” queries.
post_id	TEXT	Identifier of the related content item.
interaction_type	TEXT	Event type: LIKE, COMMENT, SHARE, etc.

Primary Key:

PRIMARY KEY (user_id, timestamp)

Clustering Order:

WITH CLUSTERING ORDER BY (timestamp DESC)

This structure ensures:

- O(1) inserts even with massive data volumes
- Instant retrieval of a user’s latest events
- Efficient slice queries (e.g., “last 50 interactions”)
- Fast filtering by interaction type (via ALLOW FILTERING)

Interaction Types:

Each row represents a discrete user action.

Equivalent to events in an event-sourcing model.

Interaction Type	Description
LIKED_POST	User liked a post
COMMENTED_POST	User commented on a post
SHARED_POST	User shared a post
FOLLOWED_USER	User followed another user
VIEWED_STORY	User viewed a story (optional)

Example Records (Cassandra Rows):

- user_id: hannah_chenoa

- post_id: post_101
 - interaction_type: SHARED_POST
 - timestamp: '2025-11-24T09:15:22.102340'
-
- user_id: prof_leobardo
 - post_id: post_102
 - interaction_type: COMMENTED_POST
 - timestamp: '2025-11-24T10:15:10.889001'

Graphic Representation (Cassandra): Wide-Column Structure

Recent Interactions for stephanie_borrego

	user_id	post_id	interaction_type	timestamp
2	stephanie_borrego	post_205	LIKED_POST	2025-11-25 15:20:10
3	stephanie_borrego	post_204	SHARED_POST	2025-11-25 13:00:45
4	stephanie_borrego	post_201	COMMENTED_POST	2025-11-25 09:10:50
5	stephanie_borrego	post_709	COMMENTED_POST	2025-11-25 09:00:00
6	stephanie_borrego	post_108	COMMENTED_POST	2025-11-24 20:25:10
7	stephanie_borrego	post_106	LIKED_POST	2025-11-24 16:55:55
8	stephanie_borrego	post_104	SHARED_POST	2025-11-24 13:30:22
9	stephanie_borrego	post_101	LIKED_POST	2025-11-24 09:35:12
10	stephanie_borrego	post_706	SHARED_POST	2025-11-10 16:00:00
11	stephanie_borrego	post_704	LIKED_POST	2025-11-02 10:00:00

Filter by interaction type

Select interaction type

COMMENTED_POST

COMMENTED_POST

LIKED_POST

SHARED_POST

1	stephanie_borrego	post_201	COMMENTED_POST	2025-11-25 09:10:50
2	stephanie_borrego	post_709	COMMENTED_POST	2025-11-25 09:00:00
3	stephanie_borrego	post_108	COMMENTED_POST	2025-11-24 20:25:10

Clear All Interactions (truncate table)

Queries

MongoDB:

Functional Requirement	Query	Justification
Unique user/email	db.users.find_one({"username": username}) db.users.find_one({"email": email})	Validation prior to user creation.

	": email}))	
View Public Profile	db.profiles.find_one({"username": username}))	Retrieves public data for display using the human-readable username.
Update Profile/Social Accounts	db.profiles.update_one({"username": username}, {"\$set": updates}) db.profiles.update_one({"username": username}, {"\$push": {"linked_social_accounts": account}}))	Updates general profile fields and adds new linked social accounts.
Account Status	db.users.update_one({"username": username}, {"\$set": {"account_status": new_status}}))	Manages account states (active, suspended, deleted). Login logic checks this status to block inactive users.
Analytics: Platform Popularity	Aggregation Pipeline: Unwinds linked_social_accounts, groups by platform_name, and sorts by count.	Provides insights on which social platforms are most frequently linked by users in the system.

Dgraph:

Functional Requirement	Query Logic (DQL)	Justification

Get Followers (FR 14)	~FOLLOWS (Reverse Edge)	Uses the reverse index on the FOLLOWS edge to instantly retrieve all users pointing to the current user.
Mutual Connections (FR 17)	uid_in(FOLLOWS, uid(target))	Efficiently filters users who are present in both the current user's FOLLOWS list and the target's FOLLOWS list.
Community Clusters (FR 18)	Nested FOLLOWS traversal	Traversals depth of three (User ->Friend ->FriendOfFriend ->User) to detect triangular “cliques” in the network.
Path Propagation (FR 15)	shortest(from: A, to: B)	Uses Dijkstra’s algorithm (built-in) to find the shortest path of influence between two users.
Influencer Detection	count(~FOLLOWS)	Aggregates the count of incoming edges to identify nodes with high centrality (Influencers).
Engagement Metrics	count(~LIKED_POST), count(~COMMENTED_POST)	Aggregates incoming interaction edges for posts authored by a user to calculate total engagement stats (Likes, Comments, Shares).

Engagement Filtering	@filter(ge(timestamp, "START") AND le(timestamp, "END"))	Applies a time-window filter to the edge metrics, allowing the dashboard to show engagement for specific periods ("Last 7 days").
Follower Growth	~FOLLOWS @facets(timestamp)	Retrieves the list of followers along with the timestamp facet stored on the edge. This data powers the "Follower Growth Over Time" line chart.

Cassandra:

Functional Requirement	Query	Justification
Record User Interaction	INSERT INTO interaction_logs (user_id, post_id, interaction_type, timestamp) VALUES (?, ?, ?, ?);	Captures every engagement event (like, comment, share) in real time. Ensures append-only event tracking for analytics.
Recent Activity Feed	SELECT user_id, post_id, interaction_type, timestamp FROM interaction_logs WHERE user_id = ? LIMIT 50;	Returns the latest interactions using the clustering key ordering (timestamp DESC). Supports "Recent Activity" dashboard.

Filter by Interaction Type	SELECT user_id, post_id, interaction_type, timestamp FROM interaction_logs WHERE user_id = ? AND interaction_type = ? ALLOW FILTERING;	Enables dashboards to show engagement types separately (likes only, comments only, etc.).
User Engagement Summary	<i>Client-side aggregation.</i> Example query: SELECT interaction_type, timestamp FROM interaction_logs WHERE user_id = ?;	Cassandra does not support server-side GROUP BY; analytics frameworks compute counts (total likes, comments, shares) client-side.
Clear User Activity Logs	TRUNCATE interaction_logs;	Used by Admin/Data Loader to reset the interaction table during testing or re-importing CSVs.

ChromaDB:

Query Function	Code Logic	Justification
Semantic Search	collection.query(query_texts=[query], n_results=3)	(Used in AI Semantic Search) Takes a user's natural language question ("What habits support well-being?"), converts it to a vector, and finds the 3 nearest neighbor posts in the vector space.

Topic Context Retrieval	collection.query(query_texts=[topic], n_results=3)	(Used in Sentiment Analytics) Retrieves posts relevant to a specific “Topic” or “Brand” context before sending them to the Hugging Face API for sentiment scoring.
Keyword Corpus Gen	collection.query(query_texts=[topic])	(Used in Keyword Extractor) Fetches a corpus of relevant text. The system then concatenates these results to feed the NLTK frequency analyzer for topic modeling.

Indexes and Optimizations

MongoDB:

A robust indexing strategy is crucial for MongoDB's performance, especially for authentication and data integrity.

Collection	Index Field	Index Type	Purpose and Optimization
users	{ "email": 1 }	Unique	Enforces FR 3 (unique email) and accelerates login queries based on email.

users	{ "username": 1 }	Unique	Enforces (unique username) and accelerates login queries and user lookups.
users	{ "account_status": 1 }	Standard	Supports by enabling efficient filtering for administrative tasks (e.g., finding all 'suspended' users).
profiles	{ "username": 1 }	Standard	Essential for fast retrieval of public profile data based on username, establishing a link with the users collection.
profiles	{ "linked_social_accounts.platform_name": 1 }	Multikey	Optimizes the aggregation pipeline to quickly scan, and group documents based on the linked social platforms.

Performance Justification

The core optimization is the **two-collection design** with identical `_ids`. This separation ensures:

1. **Security:** The sensitive users collection is accessed only during authentication.
2. **Performance:** The profiles collection, containing public display data, can be accessed instantly for public profile views without involving the sensitive authentication collection.

Cassandra:

Column	Index Type (Cassandra Mechanism)	Purpose and Optimization
<code>user_id</code>	Partition Key	Partitions all events per user. Ensures constant-time lookup of a user's entire activity history. Enables FR 20-22 ("View Activity Feed" and "Personal Interaction Timeline").
<code>timestamp</code>	Clustering Key (DESC)	Orders events from newest to oldest inside each partition. Supports fast retrieval of recent interactions without full-table scans.
<code>interaction_type</code>	Filtering (ALLOW FILTERING)	Used for optional dashboards to filter "likes", "comments", "shares". Not used as an index; Cassandra reads small partitions, so filter cost is minimal.
<code>post_id</code>	Regular Column	Included for completeness to allow displaying target content for each interaction. Stored with minimal overhead inside the partition.

Dgraph:

Predicate	Index Type	Purpose
user_id	@index(exact)	Enables lookup of user nodes by their string ID.
post_id	@index(exact)	Enables lookup of post nodes.
timestamp	@index(hour)	Allows filtering interactions by time ("Interactions in the last 24h") without scanning the whole graph.
relationship_strength	@index(float)	Allows sorting friends by "Closeness" to prioritize content in the feed.
FOLLOWS	@reverse	Automatically maintains a reverse index, allowing the system to query "Who follows me?" as efficiently as "Who do I follow?".

Performance Justification

Dgraph was selected over relational databases for this module because social traversals (Friends of Friends) require expensive recursive JOIN operations in SQL. Dgraph stores relationships as physical links on disk, allowing for constant time per

hop traversal. By using Facets for timestamps instead of intermediate interaction nodes, we reduce the graph size and traversal depth, ensuring real-time responsiveness even as the dataset grows.

Chromadb:

To ensure high performance for semantic queries, the following optimizations are inherent in the ChromaDB implementation.

- **HNSW (Hierarchical Navigable Small World) Index:**
 - **Justification:** ChromaDB automatically builds an HNSW index for the vector data. This graph-based index allows for Approximate Nearest Neighbor (ANN). This is significantly faster than a brute-force scan, ensuring the “AI Search” responds instantly even as the dataset grows.
- **Automatic Embedding Generation:**
 - **Justification:** The system uses the default all-MiniLM-L6-v2 model for embeddings. This model is optimized for speed and performance on standard CPUs (Latent usage), removing the dependency on external GPU-heavy APIs (like OpenAI) for the core vectorization task.
- **Metadata Linking (post_id):**
 - **Justification:** By strictly storing the post_id as the ID of the vector document, the system maintains a **relational link**. After a semantic search finds relevant content, the application can instantly fetch the author or engagement metrics for that specific post from Dgraph using this shared ID.

Conclusion

As a team, we developed a Social Media Analytics Tool that meets its objective by combining various database technologies and AI components in a manner that aligns with the real demands of modern platforms. Using MongoDB for secure profile handling, Cassandra for high-volume event logs, Dgraph for relationship analysis, and a vector engine with ChromaDB and Hugging Face allowed us to manage each type of data effectively and keep the system responsive. Integrating semantic search and sentiment analysis was the most challenging part, since it pushed us into deeper

NLP and embedding techniques, but it also added the highest value to the platform. The biggest technical hurdle was making all these systems communicate reliably while maintaining performance and consistency, and it's an area we still need to refine for a cleaner long-term solution. Overall, the project resulted in a scalable, secure, and future-oriented tool capable of going beyond basic analytics to uncover meaningful patterns in user interactions and community behavior.

Sources of Information

- GeeksforGeeks. (2025, July 23). Sentiment Analysis using HuggingFace's RoBERTa Model. GeeksforGeeks. <https://www.geeksforgeeks.org/nlp/sentiment-analysis-using-huggingfaces-roberta-model/>
- Mhadhbi, N. (2024, September 29). Streamlit Python: Tutorial. <https://www.datacamp.com/tutorial/streamlit>
- Search and Filtering - GraphQL - Documentation - Discuss DGraph. (2020, August 28). Discuss Dgraph. <https://discuss.dgraph.io/t/search-and-filtering-graphql/9761>
- DataMListic. (2024, May 22). Vector Database Search - Hierarchical Navigable Small Worlds (HNSW) explained [Video]. YouTube. <https://www.youtube.com/watch?v=77QH0Y2PYKg>
- Ishimwe, H. (2025, April 23). How to create vector embeddings & Automatically update them in PostgreSQL. Tiger Data Blog. <https://www.tigerdata.com/blog/how-to-automatically-create-update-embeddings-in-postgresql>
- Inera. (2024, March 21). Metadata and linking | Inera. <https://inera.com/metadata-and-linking/>
- Yiannis Pitsillides. (2024, December 23). Streamlit App for Predicting Customer Churn | Live Predictions with Pretrained Python ML Model [Video]. YouTube. <https://www.youtube.com/watch?v=YKFleQ0QJNU>
- Whitenton, K. (2024, October 24). Filters vs. Facets: Definitions. Nielsen Norman Group. <https://www.nngroup.com/articles/filters-vs-facets/>
- Facets - Dgraph. (n.d.). Hypermode. <https://docs.hypermode.com/dgraph/concepts/facets>